CSS Flexbox Align-Items

Complete Guide: From Basics to Advanced

Contents

1 Introduction **2**
- 1.1 What is align-items? . . . 2
- 1.2 Why Use align-items? . . . 2
- 1.3 Main Axis vs. Cross Axis . . . 2

2 All align-items Values **3**
- 2.1 Value Overview . . . 3
- 2.2 Comparison Table . . . 3

3 Core Values **4**
- 3.1 stretch (Default) . . . 4
  - 3.1.1 Syntax . . . 4
  - 3.1.2 Characteristics . . . 4
  - 3.1.3 Examples . . . 4
- 3.2 flex-start . . . 6
  - 3.2.1 Syntax . . . 6
  - 3.2.2 Characteristics . . . 7
  - 3.2.3 Examples . . . 7
- 3.3 flex-end . . . 9
  - 3.3.1 Syntax . . . 9
  - 3.3.2 Characteristics . . . 9
  - 3.3.3 Examples . . . 9
- 3.4 center . . . 11
  - 3.4.1 Syntax . . . 11
  - 3.4.2 Characteristics . . . 11
  - 3.4.3 Examples . . . 12
- 3.5 baseline . . . 14
  - 3.5.1 Syntax . . . 14
  - 3.5.2 Characteristics . . . 15
  - 3.5.3 Examples . . . 15

4 Combining align-items with Other Properties **17**
- 4.1 Perfect Centering Pattern . . . 17
- 4.2 align-items with flex-direction . . . 17
- 4.3 align-items with align-self . . . 18

5 Practical Applications **19**
5.1 Application 1: Card Layout 19
5.2 Application 2: Header 20
5.3 Application 3: Form Layout 21

6 Responsive align-items **23**
6.1 Mobile-First Approach 23
6.2 Responsive Navigation 23

7 Troubleshooting **25**
7.1 Issue 1: align-items Not Working 25
7.2 Issue 2: Items Not Aligning as Expected 25
7.3 Issue 3: Items Stretching Unwanted 26
7.4 Issue 4: Mixed Content Heights 26

8 Best Practices **27**
8.1 When to Use Each Value 27
8.2 Common Patterns 27
8.3 Accessibility Tips 28

9 Summary: Key Takeaways **29**
9.1 The Five Values 29
9.2 Key Rules 29
9.3 Quick Reference 29
9.4 Main Axis vs. Cross Axis 30
9.5 Comparison: align-items Values 31

Chapter 1

Introduction

1.1 What is align-items?

The `align-items` property controls how flex items are aligned along the cross axis. While `justify-content` aligns items on the main axis, `align-items` handles alignment perpendicular to that.

Key Point: `align-items` works on the cross axis. The direction of the cross axis depends on `flex-direction` (vertical for row, horizontal for column).

1.2 Why Use align-items?

- Vertically center content easily
- Stretch items to fill container height
- Align items to top or bottom
- Create professional layouts
- Handle dynamic content gracefully
- Works perfectly with justify-content
- No manual margin calculations needed
- Perfect for responsive design

1.3 Main Axis vs. Cross Axis

Remember the difference:

- **justify-content:** Aligns on main axis
- **align-items:** Aligns on cross axis
- With `flex-direction: row` → Cross axis is vertical
- With `flex-direction: column` → Cross axis is horizontal

Chapter 2

All align-items Values

2.1 Value Overview

`align-items` has six main values:

1. `stretch` - Items stretch to fill container (default)
2. `flex-start` - Items aligned to start of cross axis
3. `flex-end` - Items aligned to end of cross axis
4. `center` - Items centered on cross axis
5. `baseline` - Items aligned to text baseline

2.2 Comparison Table

Value	**Behavior**
stretch	Items grow to fill available space
flex-start	Items at start of cross axis
flex-end	Items at end of cross axis
center	Items centered on cross axis
baseline	Items aligned to text baseline

Chapter 3

Core Values

3.1 stretch (Default)

Items stretch to fill the container along the cross axis.

3.1.1 Syntax

```
.flex-container {
    display: flex;
    align-items: stretch;  /* Default */
}
```

3.1.2 Characteristics

- Items grow to fill container height (or width for column)
- Default behavior when no height is set
- Perfect for equal-height layouts
- Useful for sidebars and columns
- Creates professional card layouts

3.1.3 Examples

Example 1: Equal Height Cards

```
<div class="cards">
    <div class="card">
        <h3>Card 1</h3>
        <p>Short content</p>
    </div>
    <div class="card">
        <h3>Card 2</h3>
        <p>This card has longer content
        that spans multiple lines and takes
        up more vertical space</p>
    </div>
```

```
    <div class="card">
        <h3>Card 3</h3>
        <p>Medium content here</p>
    </div>
</div>

.cards {
    display: flex;
    align-items: stretch;  /* Cards stretch to same height */
    gap: 20px;
    min-height: 300px;
}

.card {
    flex: 1;
    background: white;
    padding: 20px;
    border-radius: 8px;
    display: flex;
    flex-direction: column;
}

.card h3 {
    margin-top: 0;
}

.card p {
    flex: 1;  /* Content fills space */
}

/* All cards same height regardless of content */
```

Example 2: Sidebar Layout

```
<div class="layout">
    <aside class="sidebar">
        <nav>
            <a href="#">Home</a>
            <a href="#">About</a>
            <a href="#">Contact</a>
        </nav>
    </aside>
    <main class="content">
        <h1>Page Title</h1>
        <p>Content here</p>
    </main>
</div>

.layout {
    display: flex;
    align-items: stretch;  /* Sidebar stretches full height */
    min-height: 100vh;
}

.sidebar {
```

```
    width: 250px;
    background: #2c3e50;
    padding: 20px;
}

.content {
    flex: 1;
    padding: 40px;
}

/* Sidebar and content stretch to full height */
```

Example 3: Image and Text

```
<div class="feature">
    <img src="feature.jpg" alt="">
    <div class="text">
        <h2>Feature Title</h2>
        <p>Description text</p>
    </div>
</div>

.feature {
    display: flex;
    align-items: stretch;
    gap: 30px;
}

.feature img {
    width: 300px;
    object-fit: cover;
    flex-shrink: 0;
}

.text {
    flex: 1;
}

/* Image stretches to match text height */
```

3.2 flex-start

Items are aligned to the start of the cross axis.

3.2.1 Syntax

```
.flex-container {
    display: flex;
    align-items: flex-start;
}
```

3.2.2 Characteristics

- Items aligned to top (for row) or left (for column)
- Items maintain their natural size
- Useful for varied-height content
- Common for component layouts
- Good for mixed content types

3.2.3 Examples

Example 1: Navigation with Logo

```
<nav class="navbar">
    <img src="logo.svg" alt="Logo" class="logo">
    <div class="nav-links">
        <a href="#">Home</a>
        <a href="#">About</a>
        <a href="#">Contact</a>
    </div>
</nav>

.navbar {
    display: flex;
    align-items: flex-start;  /* Logo at top */
    gap: 20px;
    padding: 10px 20px;
    background: #2c3e50;
}

.logo {
    width: 40px;
    height: 40px;
}

.nav-links {
    display: flex;
    flex-direction: column;
    gap: 8px;
}

.nav-links a {
    color: white;
    text-decoration: none;
}

/* Logo at top, links below */
```

Example 2: Card with Icon

```
<div class="card">
    <div class="icon "></div>
    <div class="content">
```

```
        <h3>Mobile First</h3>
        <p>Build responsive designs that work
        on all devices. Start with mobile
        and scale up.</p>
    </div>
</div>

.card {
    display: flex;
    align-items: flex-start;  /* Icon at top */
    gap: 15px;
    padding: 20px;
    background: white;
    border-radius: 8px;
}

.icon {
    font-size: 32px;
    flex-shrink: 0;
}

.content h3 {
    margin-top: 0;
}

/* Icon aligned to top of text */
```

Example 3: List with Avatars

```
<div class="user-list">
    <div class="user-item">
        <img src="user1.jpg" alt="" class="avatar">
        <div class="user-info">
            <h4>John Doe</h4>
            <p>@johndoe</p>
            <p>This is a longer bio that might
            span multiple lines</p>
        </div>
    </div>
</div>

.user-list {
    display: flex;
    flex-direction: column;
    gap: 15px;
}

.user-item {
    display: flex;
    align-items: flex-start;  /* Avatar at top */
    gap: 15px;
    padding: 15px;
    background: white;
    border-radius: 8px;
}
```

```
.avatar {
    width: 50px;
    height: 50px;
    border-radius: 50%;
    flex-shrink: 0;
}

.user-info h4 {
    margin: 0 0 5px 0;
}

/* Avatar aligned to top of content */
```

3.3 flex-end

Items are aligned to the end of the cross axis.

3.3.1 Syntax

```
.flex-container {
    display: flex;
    align-items: flex-end;
}
```

3.3.2 Characteristics

- Items aligned to bottom (for row) or right (for column)
- Items maintain their natural size
- Useful for baseline alignment
- Good for footer content
- Creates unique visual effects

3.3.3 Examples

Example 1: Footer with Logo and Text

```
<footer class="footer">
    <div class="company-info">
        <h3>Company Name</h3>
        <p>Building great things</p>
    </div>
    <img src="logo.svg" alt="Logo" class="logo">
</footer>

.footer {
    display: flex;
    align-items: flex-end;  /* Logo at bottom */
```

```
    justify-content: space-between;
    gap: 30px;
    padding: 40px;
    background: #2c3e50;
    color: white;
}

.logo {
    width: 100px;
    height: 100px;
}

.company-info h3 {
    margin: 0;
}

/* Logo aligned to bottom */
```

Example 2: Search Bar with Button

```
<div class="search-box">
    <input type="text" placeholder="Search...">
    <button>Search</button>
</div>

.search-box {
    display: flex;
    align-items: flex-end;  /* Button at bottom */
    gap: 10px;
}

.search-box input {
    flex: 1;
    padding: 12px;
    border: 1px solid #ddd;
    border-radius: 4px;
    font-size: 14px;
}

.search-box button {
    padding: 12px 24px;
    background: #3498db;
    color: white;
    border: none;
    border-radius: 4px;
    cursor: pointer;
    white-space: nowrap;
}

/* Button aligns to input baseline */
```

Example 3: Price Display

```
<div class="price-display">
    <span class="currency">$</span>
```

```
    <span class="amount">29</span>
    <span class="period">/month</span>
</div>

.price-display {
    display: flex;
    align-items: flex-end;  /* Align to bottom */
    gap: 5px;
}

.price-display .currency {
    font-size: 16px;
}

.price-display .amount {
    font-size: 48px;
    font-weight: bold;
}

.price-display .period {
    font-size: 14px;
}

/* Price aligned visually correct */
```

3.4 center

Items are centered on the cross axis.

3.4.1 Syntax

```
.flex-container {
    display: flex;
    align-items: center;
}
```

3.4.2 Characteristics

- Items centered vertically (for row) or horizontally (for column)
- Most common align-items value
- Perfect centering with justify-content: center
- Great for symmetric layouts
- Very popular for modern designs

3.4.3 Examples

Example 1: Perfect Centering

```
<div class="hero">
    <div class="hero-content">
        <h1>Welcome</h1>
        <p>This content is perfectly centered</p>
        <button>Get Started</button>
    </div>
</div>

.hero {
    display: flex;
    justify-content: center;  /* Horizontal center */
    align-items: center;       /* Vertical center */
    min-height: 500px;
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
}

.hero-content {
    text-align: center;
    color: white;
}

/* Content perfectly centered both ways */
```

Example 2: Icon with Label

```
<div class="feature">
    <div class="icon "></div>
    <span class="label">Fast</span>
</div>

.feature {
    display: flex;
    align-items: center;  /* Icon and text centered */
    gap: 10px;
}

.icon {
    font-size: 32px;
    flex-shrink: 0;
}

.label {
    font-weight: bold;
}

/* Icon and text vertically aligned */
```

Example 3: Checkbox with Label

```
<div class="checkbox-group">
    <input type="checkbox" id="terms">
    <label for="terms">I agree to the terms and conditions.
```

```
    This can be a longer text that might span
    multiple lines.</label>
</div>

.checkbox-group {
    display: flex;
    align-items: center;  /* Checkbox centered with text */
    gap: 10px;
}

.checkbox-group input[type="checkbox"] {
    width: 20px;
    height: 20px;
    flex-shrink: 0;
    cursor: pointer;
}

.checkbox-group label {
    cursor: pointer;
    line-height: 1.5;
}

/* Checkbox centered vertically with label */
```

Example 4: Button with Icon

```
<button class="button-with-icon">
    <span class="icon "></span>
    <span>Download</span>
</button>

.button-with-icon {
    display: flex;
    align-items: center;  /* Icon and text centered */
    gap: 8px;
    padding: 12px 24px;
    background: #3498db;
    color: white;
    border: none;
    border-radius: 4px;
    cursor: pointer;
}

.button-with-icon .icon {
    font-size: 18px;
}

/* Icon and text perfectly aligned */
```

Example 5: Navigation Bar

```
<nav class="navbar">
    <div class="logo">Logo</div>
    <div class="nav-links">
        <a href="#">Home</a>
```

```
        <a href="#">About</a>
        <a href="#">Contact</a>
    </div>
    <button class="login">Login</button>
</nav>

.navbar {
    display: flex;
    align-items: center;  /* All items vertically centered */
    justify-content: space-between;
    height: 60px;
    padding: 0 30px;
    background: #2c3e50;
}

.logo {
    color: white;
    font-weight: bold;
    font-size: 20px;
}

.nav-links {
    display: flex;
    gap: 20px;
}

.nav-links a {
    color: white;
    text-decoration: none;
}

.login {
    background: #3498db;
    color: white;
    border: none;
    padding: 8px 16px;
    border-radius: 4px;
    cursor: pointer;
}

/* All navigation items centered vertically */
```

3.5 baseline

Items are aligned to their text baseline.

3.5.1 Syntax

```
.flex-container {
    display: flex;
    align-items: baseline;
}
```

3.5.2 Characteristics

- Items aligned to text baseline
- Useful for text-heavy layouts
- Creates natural reading alignment
- Accounts for different font sizes
- Less common but very useful

3.5.3 Examples

Example 1: Mixed Font Sizes

```
<div class="text-group">
    <h1>35</h1>
    <span class="unit">°C</span>
    <span class="label">Temperature</span>
</div>

.text-group {
    display: flex;
    align-items: baseline;  /* Align to baseline */
    gap: 10px;
}

.text-group h1 {
    margin: 0;
    font-size: 48px;
}

.text-group .unit {
    font-size: 18px;
}

.text-group .label {
    font-size: 12px;
    color: #666;
}

/* All text aligned naturally by baseline */
```

Example 2: Price with Currency

```
<div class="price">
    <span class="currency">$</span>
    <span class="amount">99</span>
    <span class="decimals">.99</span>
</div>

.price {
    display: flex;
    align-items: baseline;
}
```

```
.price .currency {
    font-size: 18px;
}

.price .amount {
    font-size: 48px;
    font-weight: bold;
}

.price .decimals {
    font-size: 24px;
    margin-left: 2px;
}

/* Text aligned naturally by baseline */
```

Chapter 4

Combining align-items with Other Properties

4.1 Perfect Centering Pattern

```
/* The ultimate centering recipe */
.perfect-center {
    display: flex;
    justify-content: center;   /* Horizontal center */
    align-items: center;        /* Vertical center */
    width: 400px;
    height: 400px;
}

/* Works with any content */
```

4.2 align-items with flex-direction

Remember that `align-items` works on the cross axis:

```
/* Row: align-items affects vertical alignment */
.horizontal {
    display: flex;
    flex-direction: row;
    align-items: center;  /* Vertical center */
}

/* Column: align-items affects horizontal alignment */
.vertical {
    display: flex;
    flex-direction: column;
    align-items: center;  /* Horizontal center */
}

/* Column with flex-end */
.column-bottom {
    display: flex;
    flex-direction: column;
```

```
    align-items: flex-end;  /* Right align */
}
```

4.3 align-items with align-self

Override `align-items` for specific items:

```
.flex-container {
    display: flex;
    align-items: center;  /* Default for all items */
}

.flex-item-top {
    align-self: flex-start;  /* Override for this item */
}

.flex-item-bottom {
    align-self: flex-end;    /* Override for this item */
}

.flex-item-stretch {
    align-self: stretch;     /* Override for this item */
}
```

Chapter 5

Practical Applications

5.1 Application 1: Card Layout

```
<div class="card">
    <img src="image.jpg" alt="" class="card-image">
    <div class="card-body">
        <h3>Card Title</h3>
        <p>Card description text</p>
    </div>
    <button class="card-button">Action</button>
</div>

.card {
    display: flex;
    flex-direction: column;
    background: white;
    border-radius: 8px;
    overflow: hidden;
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}

.card-image {
    width: 100%;
    height: 200px;
    object-fit: cover;
}

.card-body {
    flex: 1;
    padding: 20px;
}

.card-button {
    padding: 12px 20px;
    background: #3498db;
    color: white;
    border: none;
    cursor: pointer;
    align-self: stretch;  /* Button full width */
```

```
}

/* Professional card layout */
```

5.2 Application 2: Header

```
<header class="header">
    <div class="logo">Logo</div>
    <nav class="nav">
        <a href="#">Home</a>
        <a href="#">About</a>
        <a href="#">Contact</a>
    </nav>
    <div class="user">
        <img src="user.jpg" alt="" class="user-avatar">
        <span>John</span>
    </div>
</header>

.header {
    display: flex;
    align-items: center;  /* All items vertically centered */
    justify-content: space-between;
    padding: 15px 30px;
    background: #2c3e50;
}

.logo {
    color: white;
    font-weight: bold;
    font-size: 20px;
}

.nav {
    display: flex;
    gap: 20px;
}

.nav a {
    color: white;
    text-decoration: none;
}

.user {
    display: flex;
    align-items: center;  /* Avatar and name centered */
    gap: 10px;
    color: white;
}

.user-avatar {
    width: 40px;
```

```
    height: 40px;
    border-radius: 50%;
}

/* Professional header with perfect alignment */
```

5.3 Application 3: Form Layout

```
<form class="form">
    <div class="form-row">
        <label for="name">Name</label>
        <input type="text" id="name" placeholder="Enter name">
    </div>
    <div class="form-row checkbox">
        <input type="checkbox" id="terms">
        <label for="terms">I agree to terms</label>
    </div>
    <button type="submit">Submit</button>
</form>

.form {
    display: flex;
    flex-direction: column;
    gap: 20px;
    max-width: 400px;
}

.form-row {
    display: flex;
    align-items: center;  /* Label and input centered */
    gap: 10px;
}

.form-row label {
    width: 100px;
    flex-shrink: 0;
}

.form-row input[type="text"] {
    flex: 1;
    padding: 10px;
    border: 1px solid #ddd;
}

.form-row.checkbox {
    align-items: flex-start;  /* Checkbox at top for long labels */
}

.form button {
    padding: 12px;
    background: #27ae60;
    color: white;
```

```
    border: none;
    border-radius: 4px;
    cursor: pointer;
}

/* Clean form layout */
```

Chapter 6

Responsive align-items

6.1 Mobile-First Approach

```
/* Mobile: Center everything */
.layout {
    display: flex;
    flex-direction: column;
    align-items: center;
}

/* Tablet: Align items to start */
@media (min-width: 768px) {
    .layout {
        flex-direction: row;
        align-items: flex-start;
    }
}

/* Desktop: Stretch for full height */
@media (min-width: 1200px) {
    .layout {
        align-items: stretch;
        min-height: 100vh;
    }
}
```

6.2 Responsive Navigation

```
/* Mobile: Centered stacked navigation */
.navbar {
    display: flex;
    flex-direction: column;
    align-items: center;
    gap: 15px;
}

/* Desktop: Horizontal centered */
```

```
@media (min-width: 768px) {
    .navbar {
        flex-direction: row;
        justify-content: space-between;
        align-items: center;
        height: 60px;
    }
}
```

Chapter 7

Troubleshooting

7.1 Issue 1: align-items Not Working

Problem: `align-items` has no effect.

Solution: Ensure container has height and `display: flex`.

```
/* WRONG: No height set */
.container {
    display: flex;
    align-items: center;  /* No visible effect */
}

/* CORRECT: Set container height */
.container {
    display: flex;
    align-items: center;
    height: 200px;  /* Now it works */
}
```

7.2 Issue 2: Items Not Aligning as Expected

Problem: Items align in unexpected direction.

Solution: Check `flex-direction` - it changes the cross axis.

```
/* Row: align-items is vertical */
.horizontal {
    display: flex;
    flex-direction: row;
    align-items: center;  /* Vertical center */
}

/* Column: align-items is horizontal */
.vertical {
    display: flex;
    flex-direction: column;
    align-items: center;  /* Horizontal center */
}
```

7.3 Issue 3: Items Stretching Unwanted

Problem: Items stretch when using default `stretch` value.
Solution: Change to `flex-start`, `center`, or `flex-end`.

```
.container {
    display: flex;
    align-items: center;  /* Items don't stretch */
}

/* Or explicitly set flex-basis */
.item {
    flex-basis: auto;  /* Respects content size */
}
```

7.4 Issue 4: Mixed Content Heights

Problem: Different sized content misaligns.
Solution: Use `baseline` for text or set explicit heights.

```
.container {
    display: flex;
    align-items: baseline;  /* Text naturally aligned */
}

/* Or set explicit height */
.item {
    height: 100px;  /* All items same height */
}
```

Chapter 8

Best Practices

8.1 When to Use Each Value

- **stretch:** Equal height layouts, sidebars, full-height designs
- **flex-start:** Icon + text combinations, cards, varied heights
- **flex-end:** Baseline alignment, footer content
- **center:** Perfect centering, balanced layouts, modern designs
- **baseline:** Mixed font sizes, price displays, text-heavy content

8.2 Common Patterns

```
/* Perfect centering */
.center-all {
    display: flex;
    justify-content: center;
    align-items: center;
    height: 200px;
}

/* Horizontal layout with centered items */
.horizontal-centered {
    display: flex;
    align-items: center;
    gap: 20px;
}

/* Card with equal height content */
.card {
    display: flex;
    flex-direction: column;
    align-items: stretch;
}

/* Icon with text */
.icon-text {
```

```
    display: flex;
    align-items: center;
    gap: 10px;
}

/* Mixed font sizes */
.mixed-text {
    display: flex;
    align-items: baseline;
    gap: 5px;
}
```

8.3 Accessibility Tips

- Maintain logical reading order
- Don't hide important content with alignment
- Ensure touch targets are at least 44px
- Test with keyboard navigation
- Consider screen reader experience

Chapter 9

Summary: Key Takeaways

9.1 The Five Values

Value	Best For
stretch	Equal height layouts
flex-start	Top/left alignment
flex-end	Bottom/right alignment
center	Perfect centering
baseline	Text alignment

9.2 Key Rules

1. Works only on cross axis
2. Cross axis depends on flex-direction
3. Requires `display: flex` to work
4. Works perfectly with justify-content
5. Can be overridden per item with align-self
6. Combine with height for best results
7. Most common: center and stretch
8. Baseline useful for text-heavy layouts
9. Responsive with media queries
10. Test at different container heights

9.3 Quick Reference

```
/* Center everything */
.center-all {
    display: flex;
```

```
    justify-content: center;
    align-items: center;
}

/* Horizontal with centered items */
.horizontal {
    display: flex;
    align-items: center;
    gap: 20px;
}

/* Vertical with centered items */
.vertical {
    display: flex;
    flex-direction: column;
    align-items: center;
}

/* Equal height */
.equal-height {
    display: flex;
    align-items: stretch;
    height: 300px;
}

/* Icon and text */
.icon-text {
    display: flex;
    align-items: center;
}

/* Baseline aligned */
.baseline {
    display: flex;
    align-items: baseline;
}
```

9.4 Main Axis vs. Cross Axis

- **justify-content:** Aligns on main axis
- **align-items:** Aligns on cross axis
- Main axis: controlled by flex-direction
- Cross axis: perpendicular to main axis
- Both needed for complete 2D alignment
- Together create professional layouts

9.5 Comparison: align-items Values

stretch	**flex-start**	**center**	**flex-end**
Full height	At top	Centered	At bottom